BOSS-KAMPF 5 · Welt 5 · DS 36 · 60 min · Hilfsmittel: Werkzeugkarten + Binärbaum-Schnittstelle (EH)

W5·Boss – Die drei Baumhüter

Wähle 2 von 3 Stationen à 12 BE. Erreichbar: 24 BE · danach Papier-Compiler · **Zusätzlich im Lab: Online-Boss „Die drei Baumhüter – Online-Duell“**
Niveau (KC 2017): nur Leistungskurs (eA) – im Grundkurs nicht abiturrelevant

Am Ende des Waldes versperren drei Baumhüter den Weg. Jeder hütet eine Baumart, die du noch nie gesehen hast: einen **Max-Heap**, einen **Huffman-Baum** und einen **B-Baum**. Alles Nötige steht im Material der jeweiligen Station. Besiege **zwei** der drei Hüter – zeig, dass du unbekannte Bäume mit deinem Wissen über Binärbäume, Rekursion und Suchbäume erschließen kannst!

Hinweise zur Bearbeitung

- Wähle **zwei** der drei Stationen A, B, C und kreuze sie auf dem Antwortbogen an. Bearbeitest du alle drei, werden die zwei zuerst angekreuzten gewertet.
- Bearbeite alle Teilaufgaben **ohne Gerät** auf dem Antwortbogen. Erlaubt sind deine Werkzeugkarten und die Binärbaum-Schnittstelle unten.
- Implementierungen in **Java** mit der Klasse BinTree der Bibliothek „Niedersachsen“: nicht generisch, getItem() liefert Object und wird **gecastet**, z. B. `int w = (Integer) b.getItem();`. Struktogramme nach den Abi-Konventionen.
- Laufzeitfehler musst du durch **explizite Abfragen** ausschließen: kein getItem, getLeft oder getRight auf einem leeren Baum. Zeitbudget: ca. 2,5 min pro BE, also ca. 30 min je Station.

Papier-Compiler

Deine Lösung zu **Station B d)** tippst du direkt nach dem Boss-Kampf **unverändert** in der Online-IDE ab und startest die Tests im Testrunner. Hast du Station B nicht gewählt, überträgst du dein Struktogramm aus **Station A d)** Zeile für Zeile nach Java (ohne inhaltliche Änderungen). Ziel: alle Tests grün – am besten schon beim **ersten Versuch**. Wer in A d) bzw. B d) jeden Zugriff auf einen leeren Baum ausschließt, hat die häufigste Abi-Falle (P1) im Griff.

Die drei Stationen

Station	Hüter	Unbekannte Baumart	Teilaufgaben	BE	gewählt
A	Die Glutwächterin	Max-Heap	a) b) c) d)	12	☐
B	Der Runenschreiber	Huffman-Baum	a) b) c) d)	12	☐
C	Die Archivarin	B-Baum	a) b) c) d)	12	☐

Hilfsmittel: Schnittstelle Binärbaum (Auszug nach EH)

Operation	Beschreibung
c BinTree() · c BinTree(o: Inhaltstyp)	erzeugt einen leeren Baum bzw. einen Baum aus einem Blatt mit Inhalt o
isEmpty(): Wahrheitswert · isLeaf(): Wahrheitswert	wahr, wenn der Baum leer ist bzw. wenn beide Teilbäume leer sind
getItem(): Inhaltstyp · setItem(o: Inhaltstyp): nichts	liefert bzw. setzt den Inhalt der Wurzel (setItem macht einen leeren Baum zum Blatt)
getLeft(): Binärbaum · getRight(): Binärbaum	liefert den linken bzw. rechten Teilbaum (ggf. einen leeren Baum, nie null)
setLeft(b: Binärbaum) · setRight(b: Binärbaum) · setEmpty()	ersetzt den linken bzw. rechten Teilbaum · macht den Baum leer

Ein nicht leerer Binärbaum hat immer zwei (ggf. leere) Teilbäume. Alle Operationen außer isEmpty, setItem und setEmpty nur auf nicht leeren Bäumen aufrufen.

Station A · Max-Heap

Die Glutwächterin

Die Glutwächterin hütet die Glutsteine des Waldes. Der heißeste Stein muss jederzeit sofort griffbereit sein – deshalb verwaltet sie die Temperaturen der Steine (in Grad) in einem **Max-Heap**, einer besonderen Art von Binärbaum (Abbildungen A.1 bis A.4).

a) Entscheiden Sie für jeden der Bäume H1, H2 und H3 aus Abbildung A.4, ob er ein Max-Heap ist. Nennen Sie gegebenenfalls die verletzte Eigenschaft. **[2 BE]**

b) In den Max-Heap aus Abbildung A.2 wird nach dem Verfahren in Abbildung A.3 der Wert 77 eingefügt. Anschließend wird aus dem entstandenen Max-Heap einmal das größte Element entfernt.

Geben Sie den Max-Heap nach dem Einfügen und nach dem Entfernen an. **[3 BE]**

c) Die Glutwächterin überlegt, die Temperaturen stattdessen in einem binären Suchbaum zu verwalten.

Vergleichen Sie Max-Heap und binären Suchbaum hinsichtlich des Zugriffs auf das größte Element und der Suche nach einem beliebigen Wert. **[2 BE]**

d) Die rekursive Operation **isMaxHeap(b: Binärbaum vom Inhaltstyp Ganzzahl): Wahrheitswert** liefert wahr, wenn der übergebene nicht leere Binärbaum b die Eltern-Max-Eigenschaft erfüllt. Die Form-Eigenschaft muss nicht geprüft werden.

Entwickeln Sie ein Struktogramm für die Operation isMaxHeap.

Begründen Sie, warum es hier – anders als bei der Prüfung eines binären Suchbaums – genügt, jeden Knoten nur mit seinen Kindknoten zu vergleichen. **[5 BE]**

Summe Station A: 12 BE

Material zu Station A

Abbildung A.1: Max-Heap

Ein Binärbaum ist ein **Max-Heap**, wenn er beide folgenden Eigenschaften erfüllt:

- **Form-Eigenschaft:** Alle Ebenen sind vollständig mit Knoten besetzt, nur die unterste Ebene darf unvollständig sein. In der untersten Ebene sind die Knoten von links nach rechts ohne Lücke angeordnet.
- **Eltern-Max-Eigenschaft:** Für jeden Knoten gilt: Sein Wert ist größer oder gleich den Werten seiner Kindknoten. (Ein Blatt erfüllt die Eigenschaft immer.)

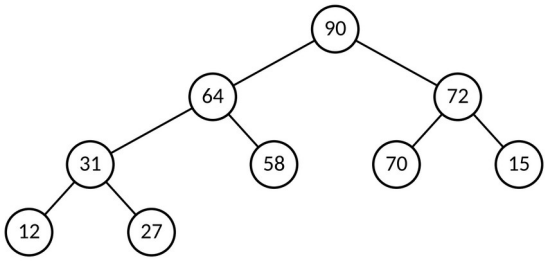
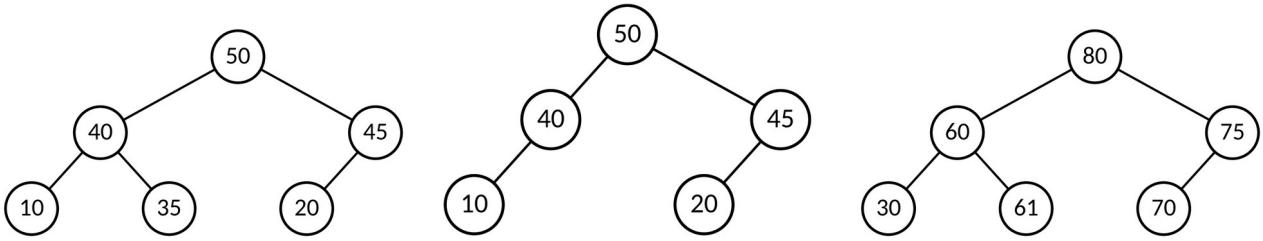


Abbildung A.2: Max-Heap der Glutwächterin (Temperaturen in Grad)

Abbildung A.3: Einfügen und Entfernen des größten Elements

Einfügen eines Werts x: (1) Erzeuge einen neuen Knoten mit x an der nächsten freien Stelle der untersten Ebene (von links nach rechts; ist die Ebene voll, ganz links in einer neuen Ebene). (2) Solange x größer ist als der Wert des Elternknotens, tausche die beiden Werte („x steigt auf“).

Entfernen des größten Elements: (1) Das größte Element steht in der Wurzel; merke es. (2) Ersetze den Wert der Wurzel durch den Wert des „letzten“ Knotens (unterste Ebene, ganz rechts) und entferne diesen Knoten. (3) Solange der neue Wert kleiner ist als der Wert mindestens eines Kindknotens, tausche ihn mit dem **größeren** der Kindwerte („versickern“).



H1 *H2* *H3*

Abbildung A.4: Drei Binärbäume

Station B · Huffman-Baum

Der Runenschreiber

Der Runenschreiber ritzt Botschaften als Folgen von kurzen (0) und langen (1) Kerben in die Rinde. Jede Kerbe kostet Kraft – häufige Zeichen sollen deshalb kurze Codewörter bekommen. Er nutzt die **Huffman-Codierung**, deren Codewörter er an einem **Codebaum** abliest (Abbildungen B.1 bis B.3).

a) Erstellen Sie nach dem Verfahren in Abbildung B.2 den Huffman-Baum für das Wort TANNENNADEL (Häufigkeiten in Abbildung B.1).

Geben Sie die Codewörter aller Zeichen und die Anzahl der Bits an, die das codierte Wort TANNENNADEL umfasst. **[4 BE]**

b) Geben Sie die Decodierung der Bitfolge **1 1 0 0 1 0 0 1 0 0 0 1 1 1** mithilfe des Codebaums aus Abbildung B.3 an. **[2 BE]**

c) Begründen Sie, dass ein Code, dessen Codewörter an einem Codebaum abgelesen werden, präfixfrei ist (kein Codewort ist der Anfang eines anderen Codeworts).

Ein Lehrling schlägt für die Zeichen aus Abbildung B.3 die Codewörter I = 0, E = 01, S = 10, R = 110 und G = 111 vor. Beurteilen Sie diesen Vorschlag. **[2 BE]**

d) Die Operation **decodiere(baum: Binärbaum vom Inhaltstyp Zeichenkette, bits: Zeichenkette): Zeichenkette** decodiert die übergebene Bitfolge mithilfe des Codebaums baum und liefert den Klartext. Die Bitfolge besteht nur aus den Zeichen '0' und '1' und ist vollständig decodierbar; der Codebaum hat mindestens zwei Blätter. Die Blätter enthalten die Zeichen als Zeichenketten der Länge 1.

Implementieren Sie die Operation decodiere in Java. **[4 BE]**

Summe Station B: 12 BE

Material zu Station B

Zeichen	D	L	T	A	E	N
Häufigkeit in TANNENNADEL	1	1	1	2	2	4

Abbildung B.1: Häufigkeiten der Zeichen im Wort TANNENNADEL (aufsteigend sortiert, bei gleicher Häufigkeit alphabetisch)

Abbildung B.2: Huffman-Verfahren (Aufbau des Huffman-Baums)

- Zu Beginn ist jedes Zeichen ein eigener Baum aus einem Blatt mit seiner Häufigkeit. Die Bäume stehen in einer Liste in der Reihenfolge aus Abbildung B.1.
- Solange die Liste mehr als einen Baum enthält: Entferne die beiden **vordersten** Bäume. Verbinde sie zu einem neuen Baum: Die Wurzel erhält die Summe der beiden Häufigkeiten, der **zuerst** entfernte Baum wird linker, der andere rechter Teilbaum. Ordne den neuen Baum so in die Liste ein, dass sie aufsteigend sortiert bleibt; er steht dabei **hinter** allen Bäumen mit gleicher Häufigkeit.
- Der letzte verbleibende Baum ist der Huffman-Baum. Das Codewort eines Zeichens ist der Weg von der Wurzel zu seinem Blatt: links = 0, rechts = 1.

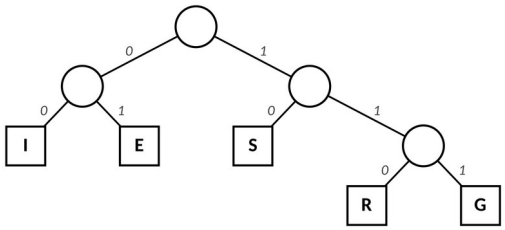


Abbildung B.3: Codebaum des Runenschreibers für die Zeichen I, E, S, R, G (Blätter: Zeichen; links = 0, rechts = 1)

Station C · B-Baum

Die Archivarin

Die Archivarin der Wurzelhalle verwaltet die Item-Nummern aller Schätze des Waldes. Das Archiv ist riesig, und jedes Öffnen einer Truhe kostet Zeit. Deshalb liegen in jeder Truhe mehrere Nummern: Sie verwendet einen **B-Baum** (Abbildungen C.1 bis C.3).

a) Beschreiben Sie, nach welchem Prinzip die Item-Nummern im B-Baum aus Abbildung C.2 angeordnet sind. **[3 BE]**

b) Geben Sie für die Suche nach der Item-Nummer 42 und nach der Item-Nummer 65 im B-Baum aus Abbildung C.2 jeweils die untersuchten Knoten an. Erläutern Sie, woran man erkennt, dass 65 nicht im Baum enthalten ist. **[2 BE]**

c) Fügen Sie nach dem Verfahren in Abbildung C.3 nacheinander die Item-Nummern 25, 45 und 15 in den B-Baum aus Abbildung C.2 ein.

Geben Sie den B-Baum nach jedem der drei Einfügevorgänge an. **[4 BE]**

d) Im großen Archiv sind 500 000 Item-Nummern in einem B-Baum der Ordnung 100 gespeichert.

Schätzen Sie ab, wie viele Knoten bei einer Suche höchstens untersucht werden müssen, und vergleichen Sie mit einem ausgeglichenen binären Suchbaum. Beurteilen Sie, unter welcher Bedingung der B-Baum trotz der vielen Vergleiche innerhalb eines Knotens vorteilhaft ist. **[3 BE]**

Summe Station C: 12 BE

Material zu Station C

Abbildung C.1: B-Baum der Ordnung M

Ein B-Baum ist ein Suchbaum, dessen Knoten mehrere Schlüssel (hier: Item-Nummern) und mehr als zwei Kindknoten haben können. Für einen B-Baum der Ordnung M gilt:

- Jeder Knoten enthält höchstens M − 1 Schlüssel. Jeder Knoten außer der Wurzel enthält mindestens $\lceil M/2 \rceil - 1$ Schlüssel ($\lceil\ \rceil$: aufrunden).
- Ein innerer Knoten mit k Schlüsseln hat genau k + 1 Kindknoten. Alle Blätter liegen auf derselben Ebene.

Abbildung C.2 zeigt einen B-Baum der Ordnung 4: höchstens 3 Schlüssel je Knoten, mindestens 1 Schlüssel je Knoten außer der Wurzel. Die Punkte markieren die Verweise auf die Kindknoten.

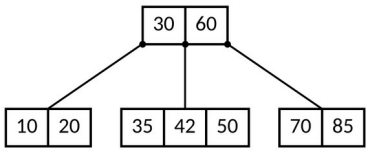


Abbildung C.2: B-Baum der Ordnung 4 mit Item-Nummern

Abbildung C.3: Einfügen in einen B-Baum der Ordnung M

- Suche wie bei einer Suche das Blatt, in das der neue Schlüssel gehört, und füge ihn dort an der passenden Stelle ein (die Schlüssel bleiben aufsteigend sortiert).
- Enthält der Knoten danach M Schlüssel (Überlauf), wird er geteilt: Der Schlüssel an Position M/2 + 1 (bei M = 4: der **dritte**) wandert in den Elternknoten und wird dort an der passenden Stelle eingefügt. Die Schlüssel davor bilden einen neuen linken, die Schlüssel danach einen neuen rechten Knoten; diese beiden Knoten werden die Kindknoten links und rechts des aufgestiegenen Schlüssels. Bei einem inneren Knoten werden die Kindknoten entsprechend mit aufgeteilt.
- Läuft dadurch der Elternknoten über, wird er ebenso geteilt. Wird die Wurzel geteilt, entsteht eine neue Wurzel, die nur den aufgestiegenen Schlüssel enthält – der Baum wächst um eine Ebene.

Antwortbogen

Gewählte Stationen: ☐ Station A (Max-Heap) ☐ Station B (Huffman-Baum) ☐ Station C (B-Baum)

Station A Die Glutwächterin – Max-Heap

A a) H1 · H2 · H3: Max-Heap? Verletzte Eigenschaft **[2 BE]**

A b) Max-Heap nach dem Einfügen von 77 · nach dem Entfernen des größten Elements **[3 BE]**

nach dem Einfügen von 77

nach dem Entfernen des größten Elements

A c) Vergleich Max-Heap – binärer Suchbaum **[2 BE]**

A d) Struktogramm isMaxHeap (Kopf mit Signatur) · Begründung **[5 BE]**

Zeichenfläche für das Struktogramm

Begründung:

Station B Der Runenschreiber – Huffman-Baum

B a) Huffman-Baum für TANNENNADEL (Zwischenschritte erwünscht) · Codewörter · Anzahl der Bits **[4 BE]**

Zeichenfläche für die Zwischenschritte und den Huffman-Baum

Zeichen	D	L	T	A	E	N
Codewort						

Anzahl der Bits für TANNENNADEL: _____

B b) Decodierung von 1 1 0 0 1 0 0 1 0 0 0 1 1 1 **[2 BE]**

B c) Präfixfreiheit · Beurteilung des Vorschlags **[2 BE]**

B d) Implementierung von decodiere – Papier-Compiler: sauber und lesbar schreiben! **[4 BE]**

Station C Die Archivarin – B-Baum

C a) Ordnungsprinzip **[3 BE]**

C b) Untersuchte Knoten bei der Suche nach 42 und nach 65 · Erläuterung **[2 BE]**

C c) B-Baum nach dem Einfügen von 25 · von 45 · von 15 **[4 BE]**

nach dem Einfügen von 25

nach dem Einfügen von 45

nach dem Einfügen von 15

C d) Abschätzung · Vergleich · Beurteilung **[3 BE]**

Auswertung (Peer-Korrektur nach Kernpunkten; gewertet werden zwei Stationen)

	a)	b)	c)	d)	Summe
Station A	a) __ / 2	b) __ / 3	c) __ / 2	d) __ / 5	__ / 12
Station B	a) __ / 4	b) __ / 2	c) __ / 2	d) __ / 4	__ / 12
Station C	a) __ / 3	b) __ / 2	c) __ / 4	d) __ / 3	__ / 12

Gesamt (zwei Stationen): _____ / 24 BE

Korrigiert von: _____ Papier-Compiler (☐ B d) ☐ A d)): ☐ grün beim 1. Versuch ☐ grün nach Korrektur ☐ nicht grün